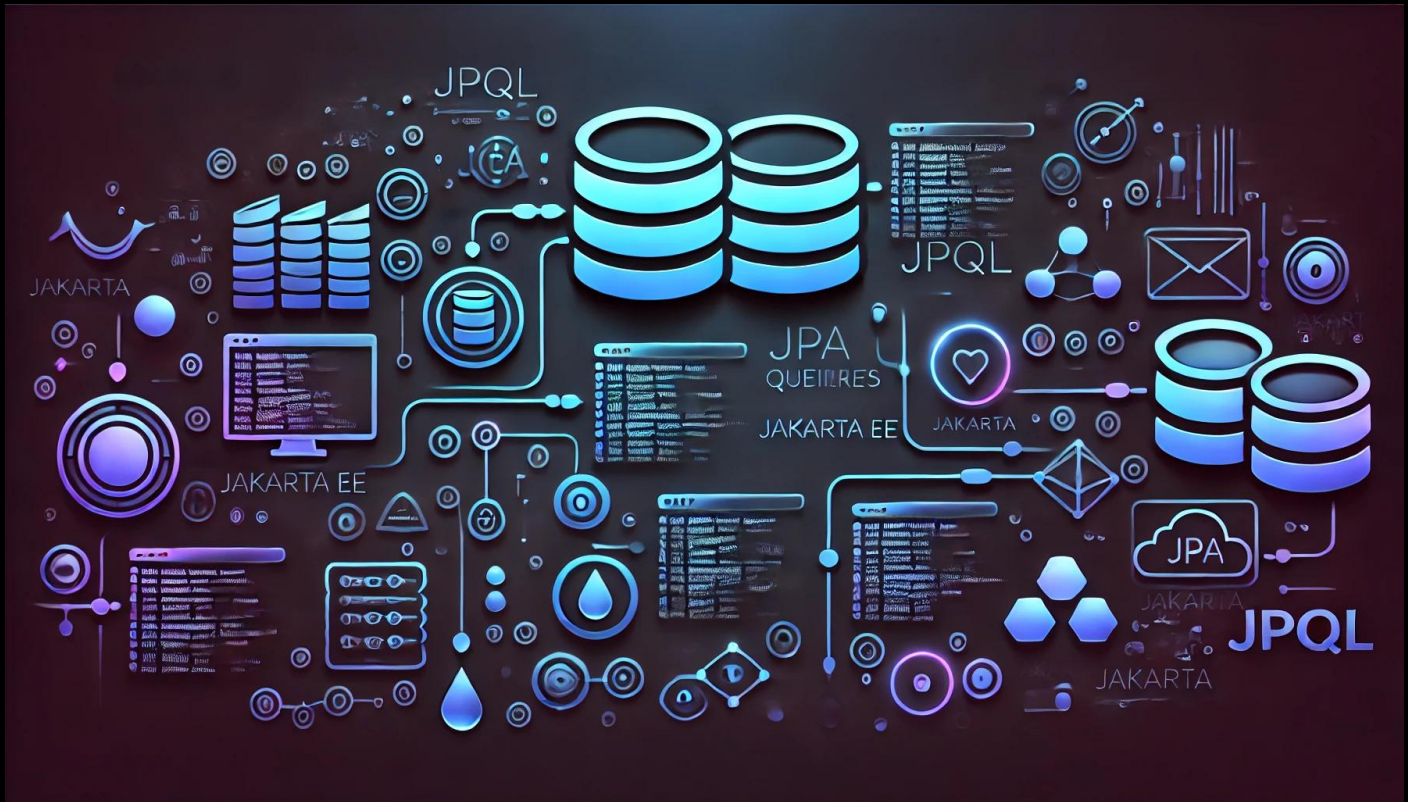


JPQL con JPA y Jakarta EE – parte 9



■ Guía: Uso de Subqueries y Joins en JPQL con JPA

1 Introducción

En esta guía aprenderemos a utilizar **Subqueries y Joins en JPQL con JPA** para realizar consultas avanzadas en bases de datos mediante Java Persistence API.

✦ Objetivos:

1. **Utilizar Subqueries** dentro de consultas JPQL para filtrar datos dinámicamente.
2. **Trabajar con Lazy Loading y Eager Loading** para optimizar la recuperación de datos en relaciones entre entidades.

◆ Casos de uso:

- ✓ Obtener datos basados en resultados de una **subconsulta**.
- ✓ Optimizar la carga de datos con **diferentes tipos de Joins** en JPA.

2 Conceptos Básicos

✚ Uso de Subqueries en JPQL

Las **subqueries** permiten realizar consultas dentro de una consulta principal, filtrando resultados de manera dinámica.

Ejemplo en JPQL:

```
SELECT p FROM Persona p WHERE p.idPersona IN  
(SELECT MIN(p1.idPersona) FROM Persona p1)
```

- ◆ Obtiene la persona con el **ID más bajo** de la tabla `Persona`.
-

✚ Lazy Loading vs Eager Loading

- ◆ **Lazy Loading (Carga Diferida)**: Los datos se cargan **solo cuando se necesitan**.
- ◆ **Eager Loading (Carga Anticipada)**: Los datos relacionados se cargan **de inmediato**.

Ejemplo con **Lazy Loading** (`JOIN`):

```
SELECT u FROM Usuario u JOIN u.persona p
```

- ◆ Recupera usuarios y **solo carga la relación con Persona si se accede a ella**.

Ejemplo con **Eager Loading** (`LEFT JOIN FETCH`):

```
SELECT u FROM Usuario u LEFT JOIN FETCH u.persona
```

- ◆ Recupera usuarios y **carga inmediatamente la relación con Persona**.
-

3 Consulta: Uso de Subqueries en JPQL

✚ Queremos obtener el objeto `Persona` con el `idPersona` más bajo utilizando una Subquery.

◆ Código actualizado

```
//14. Uso de subquery  
log.info("\n14. Uso de Subqueries");
```

```
jpql = "SELECT p FROM Persona p WHERE p.idPersona IN "
      + "(SELECT MIN(p1.idPersona) FROM Persona p1)";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);
```

🗉 Explicación:

- La **subquery** (SELECT MIN(p1.idPersona) FROM Persona p1) obtiene el idPersona más bajo.
- La consulta principal recupera la Persona con ese idPersona.

4 Consulta: Uso de JOIN con Lazy Loading

✂ Queremos obtener Usuario y su relación con Persona utilizando Lazy Loading.

◆ Código actualizado

```
//15. Uso de join con lazy loading
log.info("\n15. Uso de JOIN con Lazy Loading");
jpql = "SELECT u FROM Usuario u JOIN u.persona p";
List<Usuario> usuarios = em.createQuery(jpql, Usuario.class).getResultList();
mostrarUsuarios(usuarios);
```

🗉 Explicación:

- JOIN relaciona la tabla Usuario con Persona, pero la **carga es diferida**.
- Si no accedemos a persona, los datos no se recuperan de la base de datos.

5 Consulta: Uso de LEFT JOIN FETCH con Eager Loading

✂ Queremos obtener Usuario y su relación con Persona utilizando Eager Loading.

◆ Código actualizado

```
//16. Uso de left join con eager loading
log.info("\n16. Uso de LEFT JOIN FETCH con Eager Loading");
jpql = "SELECT u FROM Usuario u LEFT JOIN FETCH u.persona";
usuarios = em.createQuery(jpql, Usuario.class).getResultList();
mostrarUsuarios(usuarios);
```

🔊 Explicación:

- `LEFT JOIN FETCH` recupera todos los usuarios y sus **relaciones con Persona en la misma consulta**.
 - **Optimiza el rendimiento** al reducir el número de consultas adicionales.
-

6 Código Completo Actualizado

```
package sga.cliente.jpql;

import java.util.List;
import jakarta.persistence.*;
import java.util.Iterator;
import sga.domain.Persona;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import sga.domain.Usuario;

public class PruebaJPQL {

    private static final Logger log = LoggerFactory.getLogger(PruebaJPQL.class);

    public static void main(String[] args) {
        String jpql = null;
        List<Persona> personas = null;

        EntityManagerFactory emf = Persistence.createEntityManagerFactory("SgaPU");
        try (EntityManager em = emf.createEntityManager()) {

            //1. Consulta de todos los objetos de tipo Persona
            log.info("\n1. Consulta de todas las Personas");
            jpql = "select p from Persona p";
            personas = em.createQuery(jpql, Persona.class).getResultList();
            mostrarPersonas(personas);

            //2. Consulta de una persona por ID
            log.info("\n2. Consulta de Persona con ID = 1");
            jpql = "SELECT p FROM Persona p WHERE p.idPersona = 1";
            Persona persona = em.createQuery(jpql, Persona.class).getSingleResult();
            log.info("Persona encontrada: " + persona);

            //3. Consulta de una persona por nombre
            log.info("\n3. Consulta de Persona por nombre 'Ivonne'");
```

```
jpql = "SELECT p FROM Persona p WHERE p.nombre = 'Ivonne'";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//4. Consulta de datos individuales (tupla)
log.info("\n4. Consulta de datos individuales: nombre, apellido, email");

jpql = "SELECT p.nombre, p.apellido, p.email FROM Persona p";
Iterator iter = em.createQuery(jpql).getResultList().iterator();

while (iter.hasNext()) {
    Object[] tupla = (Object[]) iter.next();
    String nombre = (String) tupla[0];
    String apellido = (String) tupla[1];
    String email = (String) tupla[2];

    log.info("Nombre: " + nombre + ", Apellido: " + apellido + ", Email: " + email);
}

//5. Obtiene el objeto Persona y el id, se crea un arreglo de tipo Object con 2 columnas
log.info("\n5. Obtiene el id y el objeto Persona, se crea un arreglo de tipo Object con 2
columnas");
Object[] tupla = null;
jpql = "select p.idPersona, p from Persona p ";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
    tupla = (Object[]) iter.next();
    int idPersona = (int) tupla[0];
    persona = (Persona) tupla[1];
    log.info("ID persona:" + idPersona);
    log.info("Objeto persona:" + persona);
}

//6. Consulta de todas las personas
log.info("\n6. Consulta de los ids de todas las personas");
jpql = "SELECT p.idPersona FROM Persona p";
List<Integer> ids = em.createQuery(jpql, Integer.class).getResultList();
ids.forEach(idPersona -> log.info("ID persona: " + idPersona));

//7. Regresa el valor minimo y maximo del idPersona (scaler result)
log.info("\n7. Regresa el valor minimo y maximo del idPersona (scaler result)");
jpql = "select MIN(p.idPersona) as MinId, MAX(p.idPersona) as MaxId, COUNT(p.idPersona) as
Contador from Persona p";
iter = em.createQuery(jpql).getResultList().iterator();
while (iter.hasNext()) {
```

```
tupla = (Object[]) iter.next();
Integer idMin = (Integer) tupla[0];
Integer idMax = (Integer) tupla[1];
Long count = (Long) tupla[2];
log.info("idMin:" + idMin + ", idMax:" + idMax + ", count:" + count);
}

//8. Cuenta los nombres de las personas que son distintos
log.info("\n8. Cuenta los nombres de las personas que son distintos");
jpql = "select COUNT(distinct p.nombre) from Persona p";
Long contador = (Long) em.createQuery(jpql).getSingleResult();
log.info("no. de personas con nombre distinto:" + contador);

//9. Concatena y convierte a mayusculas el nombre y apellido
log.info("\n9. Concatena y convierte a mayusculas el nombre y apellido");
jpql = "select CONCAT(p.nombre, ' ', p.apellido) as Nombre from Persona p";
List<String> nombres = em.createQuery(jpql).getResultList();
for (String nombreCompleto : nombres) {
    log.info(nombreCompleto);
}

//10. Obtiene el objeto persona con id igual al parametro proporcionado
log.info("\n10. Obtener objeto Persona por ID");
int idPersona = 1; // Podemos cambiar este valor dinámicamente
// Definir consulta con parámetro
jpql = "SELECT p FROM Persona p WHERE p.idPersona = :id";
Query consulta = em.createQuery(jpql);
consulta.setParameter("id", idPersona);
// Ejecutar consulta y obtener resultado único
persona = (Persona) consulta.getSingleResult();
log.info("Persona encontrada: " + persona);

//11. Obtiene las personas que contengan una letra a en el nombre, sin importar si es
mayusculas o minuscula
log.info("\n11. Obtener personas con letra 'a' en el nombre (ignorar
mayusculas/minusculas)");
// Definir consulta con LIKE y UPPER()
jpql = "SELECT p FROM Persona p WHERE UPPER(p.nombre) LIKE UPPER(:parametro)";
String parametroString = "%a%"; // Busca la letra 'a' en cualquier posición
consulta = em.createQuery(jpql);
consulta.setParameter("parametro", parametroString);
personas = consulta.getResultList();
mostrarPersonas(personas);

//12. Uso de between
```

```

log.info("\n12. Filtrar registros con BETWEEN");
jpql = "SELECT p FROM Persona p WHERE p.idPersona BETWEEN 1 AND 10";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//13. Uso del ordenamiento
log.info("\n13. Ordenar registros con ORDER BY");
jpql = "SELECT p FROM Persona p WHERE p.idPersona > 1 ORDER BY p.nombre DESC, p.apellido
DESC";

personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//14. Uso de subquery
log.info("\n14. Uso de Subqueries");
jpql = "SELECT p FROM Persona p WHERE p.idPersona IN "
      + "(SELECT MIN(p1.idPersona) FROM Persona p1)";
personas = em.createQuery(jpql, Persona.class).getResultList();
mostrarPersonas(personas);

//15. Uso de join con lazy loading
log.info("\n15. Uso de JOIN con Lazy Loading");
jpql = "SELECT u FROM Usuario u JOIN u.persona p";
List<Usuario> usuarios = em.createQuery(jpql, Usuario.class).getResultList();
mostrarUsuarios(usuarios);

//16. Uso de left join con eager loading
log.info("\n16. Uso de LEFT JOIN FETCH con Eager Loading");
jpql = "SELECT u FROM Usuario u LEFT JOIN FETCH u.persona";
usuarios = em.createQuery(jpql, Usuario.class).getResultList();
mostrarUsuarios(usuarios);
}
}

private static void mostrarPersonas(List<Persona> personas) {
    for (Persona p : personas) {
        log.info("Persona: " + p);
    }
}

private static void mostrarUsuarios(List<Usuario> usuarios) {
    for (Usuario u : usuarios) {
        log.info("Usuario: " + u);
    }
}
}

```

}

7 Resultado Esperado

Si ejecutamos la clase `PruebaJPQL`, veremos en la consola:

14. Uso de Subqueries

```
[EL Fine]: sql: ServerSession(589311950)--Connection(620423816)--SELECT t0.id_persona, t0.APELLIDO, t0.EMAIL,
t0.NOMBRE, t0.TELEFONO FROM PERSONA t0 WHERE t0.id_persona IN (SELECT MIN(t1.id_persona) FROM PERSONA t1)
[main] INFO sga.cliente.jpql.PruebaJPQL - Persona: Persona{idPersona=1, nombre=Rafael, apellido=Juarez,
email=rjuarez@mail.com, telefono=55667788}
[main] INFO sga.cliente.jpql.PruebaJPQL -
```

15. Uso de JOIN con Lazy Loading

```
[EL Fine]: sql: ServerSession(589311950)--Connection(620423816)--SELECT t1.id_usuario, t1.PASSWORD,
t1.USERNAME, t1.id_persona FROM PERSONA t0, USUARIO t1 WHERE (t0.id_persona = t1.id_persona)
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=1, username=rguzman, password=123,
persona=Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}}
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=2, username=ilara, password=123,
persona=Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}}
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=4, username=hhernandez, password=123,
persona=Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}}
[main] INFO sga.cliente.jpql.PruebaJPQL -
```

16. Uso de LEFT JOIN FETCH con Eager Loading

```
[EL Fine]: sql: ServerSession(589311950)--Connection(620423816)--SELECT t1.id_usuario, t1.PASSWORD,
t1.USERNAME, t1.id_persona, t0.id_persona, t0.APELLIDO, t0.EMAIL, t0.NOMBRE, t0.TELEFONO FROM USUARIO t1 LEFT
OUTER JOIN PERSONA t0 ON (t0.id_persona = t1.id_persona)
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=1, username=rguzman, password=123,
persona=Persona{idPersona=1, nombre=Rafael, apellido=Juarez, email=rjuarez@mail.com, telefono=55667788}}
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=2, username=ilara, password=123,
persona=Persona{idPersona=2, nombre=Ivonne, apellido=Lara, email=ilara@mail.com, telefono=55443322}}
[main] INFO sga.cliente.jpql.PruebaJPQL - Usuario: Usuario{idUsuario=4, username=hhernandez, password=123,
persona=Persona{idPersona=5, nombre=Hugo, apellido=Hernandez, email=hhernandez@mail.com, telefono=55778822}}
```

8 Conclusión

🎯 En esta guía aprendimos a utilizar **Subqueries y Joins en JPQL con JPA** para mejorar nuestras consultas y optimizar la carga de datos.

- ◆ **Puntos clave:**
- ✓ **Subqueries** permiten consultas dinámicas avanzadas.
- ✓ **Lazy Loading** carga datos solo cuando se necesitan.
- ✓ **Eager Loading** carga datos inmediatamente para optimizar rendimiento.

🚀 ¡Ahora puedes aplicar estas técnicas en tus proyectos!

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)